

Deliver Multi-City Weather Forecasts from OpenWeatherMap to Gmail Inbox

An intelligent n8n workflow that automatically generates professional, color-coded weather forecast emails for multiple cities using OpenWeatherMap data and GPT-4 AI composition.

Overview

This workflow automates weather forecast delivery by collecting city names, fetching 5 day forecasts from OpenWeatherMap, and generating professionally formatted HTML emails using GPT-4. The AI creates condition-based color-coded reports with safety precautions and sends them via Gmail.

Features

- **Multi-City Support:** Process up to 3 cities simultaneously
- **Intelligent Geocoding:** Automatic city name resolution to coordinates
- **AI-Powered Formatting:** GPT-4 generates professional HTML emails with contextual advice
- **Color-Coded Alerts:**
 - o Light yellow for clear weather
 - o Light blue for cloudy conditions
 - o Red for rain
 - o Amber for extreme heat ($\geq 35^{\circ}\text{C}$)
- **Safety Warnings:** Automated precautions for heavy rain, extreme heat, high winds, and thunderstorms
- **5-Day Forecasts:** Comprehensive daily summaries including temperature, precipitation, wind speed, and humidity

How It Works

The workflow begins with a form trigger accepting up to three city names. Each city is geocoded through OpenWeatherMap's API to obtain coordinates. The workflow then retrieves 5-day forecasts and processes the raw data into daily summaries including temperature ranges, precipitation levels, wind speeds, and weather conditions.

JavaScript nodes aggregate forecast data by day, calculating minimum and maximum temperatures, dominant weather conditions, and precipitation probabilities. This structured data is passed to GPT-4, which generates an HTML email with intelligent formatting light yellow

backgrounds for clear days, light blue for clouds, red for rain, and amber for extreme heat conditions.

The AI assistant adds contextual safety warnings for heavy rain, extreme heat, high winds, and thunderstorms. A validation node ensures proper JSON formatting before Gmail sends the final briefing.

Workflow Architecture

1. Input & Normalization

- Form Trigger: Collects 3 city names from users
- JS - Normalize City Inputs: Filters metadata and trims city names

2. Geocoding (OWM)

- HTTP - OWM Geocoding: Resolves city names to lat/lon coordinates using OpenWeatherMap Geocoding API

3. Pairing Cities ↔ Coordinates

- MERGE - Pair Cities: Aligns normalized cities with geocode data by position
- JS - Pick Lat/Lon: Extracts asked_city, country, lat, lon
- SET - Carry Asked City to Forecast: Preserves original city name through pipeline

4. Forecast Retrieval

- OpenWeatherMap (OWM) - 5 Day Forecast: Fetches 5-day weather data per city

5. Summarization & Packaging

- JS - Build Daily Summaries (5 days): Aggregates raw forecast data into daily summaries with:
 - o Min/max temperatures
 - o Dominant weather conditions
 - o Rain totals and precipitation probability
 - o Wind speeds
 - o Humidity averages

- JS - Bundle Cities for LLM: Packages all city data into single payload

6. LLM Weather Briefing

- LLM - AI Weather Briefing Composer: GPT-4 generates professional HTML email with:
 - o Condition-based row coloring
 - o Weather emojis and formatted tables
 - o Safety precautions and general tips
 - o Engaging subject lines
- JS - Validate JSON from LLM: Extracts and validates subject/html from AI response

7. Delivery

- **EMAIL - Send Weather Briefing:** Sends formatted email to recipients

Requirements

API Credentials

- OpenWeatherMap API Key: Get free key
- OpenAI API Key: Get API access
- Gmail OAuth2: Gmail account with OAuth2 authentication

n8n Setup

- n8n instance (cloud or self-hosted)
- n8n version 1.0+ recommended

Installation & Setup

1. Import Workflow

1. Copy the workflow JSON from the repository
2. In n8n, click "Import from File" or paste JSON directly
3. The workflow will be imported with all nodes and connections

2. Configure Credentials

OpenWeatherMap API

1. Sign up at OpenWeatherMap
2. Generate a free API key

3. In n8n, go to Credentials → Create New → OpenWeatherMap API
4. Enter your API key
5. Assign to the OpenWeatherMap (OWM) - 5 Day Forecast node

OpenAI API

1. Get API key from OpenAI Platform
2. In n8n, create OpenAI API credential
3. Assign to LLM - AI Weather Briefing Composer node

Gmail OAuth2

1. In n8n, create Gmail OAuth2 credential
2. Follow n8n's OAuth setup wizard
3. Authorize your Gmail account
4. Assign to GMAIL - Send Weather Briefing node

3. Update Recipient Email

In the **GMAIL - Send Weather Briefing** node, change the sendTo field to your desired recipient email address.

4. Activate Workflow

1. Toggle the workflow to Active
2. Access the form URL from the TRIGGER - Input City Names node
3. Test by submitting 3 city names

Usage

Via Web Form

1. Navigate to the form URL (found in TRIGGER node)
2. Enter up to 3 city names
3. Submit the form
4. Receive formatted weather email within 30-60 seconds

Use Cases

- **Field Operations Management:** Daily weather briefings for construction, utility workers, or agricultural teams across multiple job sites
- **Travel Planning:** Automated weather reports for upcoming trips to help with packing and itinerary adjustments
- **Event Management:** Weather advisories for outdoor events, weddings, or festivals with actionable recommendations
- **Logistics & Transportation:** Route planning support with precipitation and wind warnings for delivery or shipping operations
- **Real Estate & Property Management:** Weather alerts for property viewings, inspections, or maintenance scheduling
- **Sports & Recreation:** Forecast notifications for coaches, athletes, or outdoor activity organizers

Troubleshooting

Common Issues

Issue: "No valid geocoding results for: [city]"

- Solution: Verify city name spelling. Try using full city names (e.g., "New York City" instead of "NYC")

Issue: OpenWeatherMap API rate limit exceeded

- Solution: Free tier allows 60 calls/minute. Add delay between requests or upgrade plan

Issue: Gmail not sending

- Solution: Verify OAuth2 connection is active. Re-authenticate if necessary

Issue: GPT-4 returns malformed JSON

- Solution: Check the JS - Validate JSON from LLM node output. The validation logic extracts JSON even from wrapped responses

Issue: Wrong city name in email (e.g., "Parliament House, Delhi" instead of "Delhi")

- Solution: This is expected behavior - OpenWeatherMap returns specific location names. The SET - Carry Asked City node preserves the original input

Performance Considerations

- **Execution Time:** ~30-60 seconds for 3 cities
- **API Costs:**
 - o OpenWeatherMap: Free tier (60 calls/min)
 - o OpenAI: ~\$0.002 per request (GPT-4o-mini)

Acknowledgments

- Built for the n8n community
- Uses OpenWeatherMap's free tier API
- Powered by OpenAI GPT-4o-mini